

Albero Binario di Ricerca (ABR)

Cos'è? : L'ABR è una osservazione di un'altra struttura dati, l'**albero binario**, ossia una struttura ricorsiva formata da una tripla (key, x, dx) questa tripla forma un **nodo** all'interno dell'albero:

- key è il valore/dato contenuto nel nodo
- x è il riferimento al nodo o al sottoalbero sinistro
- dx è il riferimento al nodo o al sottoalbero destro

La differenza tra gli alberi normali e gli ABR è che il sottoalbero x conterrà valori che sono strettamente minori della radice, mentre quello dx conterrà valori strettamente maggiori rispetto alla radice

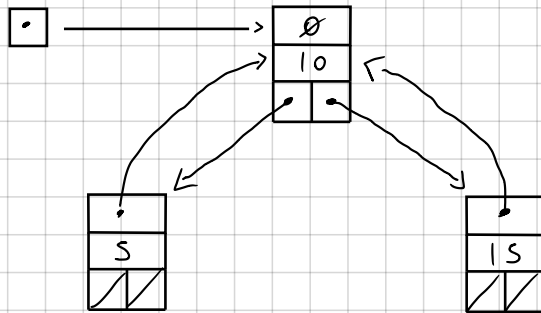
Perché usarlo? : Se implementato correttamente è una delle strutture dati più efficienti per tempi di calcolo

	Aggiunta	Correzione	Ricerca
Array	$O(1)$	$O(n)$	$O(n)$
Array ordinato	$O(n)$	$O(n)$	$O(\lg(n))$
Lista	$O(1)$ <i>in testa</i>	$O(n)$	$O(n)$
Lista ordinata	$O(n)$	$O(n)$	$O(n)$
ABR bilanciato	$O(\lg(n))$	$O(\lg(n))$	$O(\lg(n))$
ABR sbilanciato	$O(n)$	$O(n)$	$O(n)$

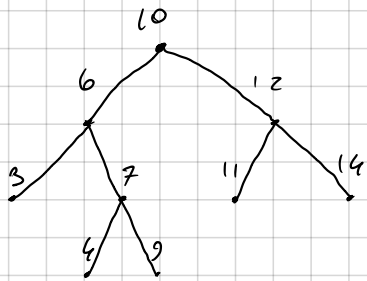
Rappresentazione in memoria

Parent	
Key	
gx	dx

T.root



Rappresentazione semplificata



procedure **BST_Search** (Nodo, x)

if nodo == null then
return null

if nodo.key == x then
return nodo

if x < nodo.key then
return BST_Search (nodo.gx, x)

else

return BST_Search (nodo.dx, x)

BST_Search (T.root, $\otimes$)

Tempo: $O(h)$

h è l'altezza dell'albero

in questo caso $h = O(\lg(n))$



in questo caso $h = O(n)$



Caso migliore: $O(1)$ (x è nella radice)

Caso peggiore: $O(h)$
(x non è presente
o sta in fondo)

ABR bilanciato = $O(\lg n)$
ABR sbilanciato = $O(n)$

Ricerca del minimo: Per le proprietà degli ABR il minimo sarà la foglia più sinistra in profondità

procedura **BST_MIN**(nodo) **BST_MIN**(T.root)

```
if (nodo == null) then // inutile cercare un albero vuoto
    return null

if (nodo.g == null) then // mi dice g sono arrivato in fondo
    return nodo

return BST_MIN(nodo.g) // continuo a scendere verso sinistra
```

Ricerca massimo: Stessa identica cosa della ricerca del minimo ma vado a destra invece che a sinistra

procedura **BST_MAX**(nodo) **BST_MAX**(T.root)

```
if (nodo == null)
    return null

if (nodo.d == null)
    return nodo

return BST_MAX(nodo.d)
```

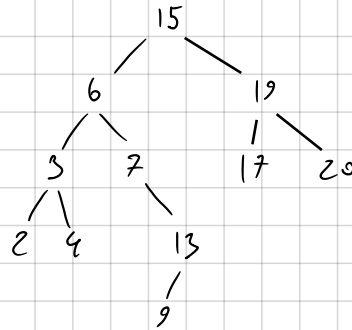
Visita di un albero

Visita simmetrica / in order : *Ordine*) Sinistra $\rightarrow$ Radice $\rightarrow$ Destra

Utilizzo) Restituisce i nodi in **ordine crescente**

procedure **BST_VISIT_INORDER** (modo)

```
if modo  $\neq$  null then  
  BST_VISIT_INORDER (modo.g)  
  print (modo.key)  
  BST_VISIT_INORDER (modo.o)
```



v. sim = 2 3 4 6 7 9 13 15 17 19 20

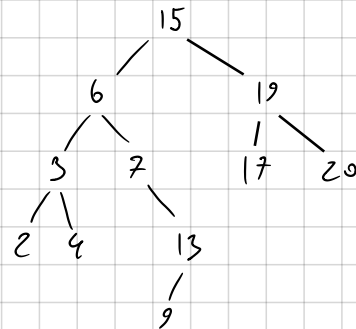
$$T(n) = \Theta(n)$$

Visita anticipata / pre-order : *Ordine*) Radice $\rightarrow$ Sinistra $\rightarrow$ Destra

Utilizzo) Salvo / copiare l'albero

procedure **BST_V_PREORDER** (modo)

```
if modo  $\neq$  null then  
  print (modo.key)  
  BST_V_PREORDER (modo.g)  
  BST_V_PREORDER (modo.o)
```



v. anticipata: 15 6 3 2 4 7 13 9 19 17 20

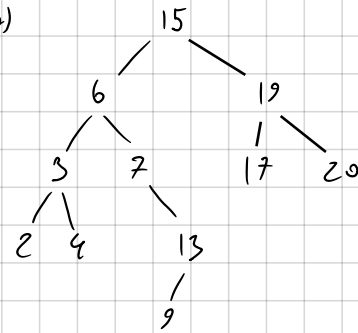
$$T(n) = \Theta(n)$$

Visita posticipata / Post-Order : Ordine : Sinistra $\rightarrow$ Destra $\rightarrow$ Radice

Utilizzo: Cancellazione di un albero

procedure **BST_V_POSTORDER** (nodo)

```
if nodo  $\neq$  null then
  BST_V_POSTORDER (nodo.gx)
  BST_V_POSTORDER (nodo.dx)
  print (nodo, Key)
```



vis. posticipata: 2 4 3 9 13 7 6 17 20 19 15

$$T(n) = \Theta(n)$$

Ricerca del predecessore / successore

A cosa serve?: Serve a trovare il valore immediatamente più piccolo (predecessore) / grande (successore) di un certo nodo

Ricerca predecessore: Dobbiamo trovare il valore più grande tra tutti quelli minori del valore del nodo dato

procedure **BST_PRED** (nodo)

```
if nodo.dx  $\neq$  null then
  return BST_MIN (nodo.dx)
```

y = nodo.p

```
while y  $\neq$  null AND y.dx = nodo.dx do
  nodo y = y
  y = y.p
```

return y

ricerca successore: Dobbiamo trovare il nodo ed valore più piccolo tra tutti quelli maggiori del nodo dato

procedure BST_SUCC (nodo)

if nodo.x $\neq$ null then

return BST_MAX (nodo.x)

y = nodo.p

While y $\neq$ null and y.x = nodo do

ndo y = y

y = y.p

return y

Implemento

Strategie: Confrontandogni volte il nodo sinistro/destro di gni livello cerchiamo di arrivare alla posizione corretta dove inserire la nostra nuova foglia

Tempi di calcolo:

procedure **BST_insert** (T, z)

$x = T.root$

$y = null$

while $x \neq null$ do

$y = x$

if $z.key < x.key$ then

$x = x.left$

else // $z.key > x.key$

$x = x.right$

$y = x.parent$

if $y == null$ then // vuol dire che l'albero è vuoto

$T.root = z$

else

if $z.key < y.key$ then

$y.left = z$

else

$y.right = z$

$z.parent = y$

Il tempo dipende da h

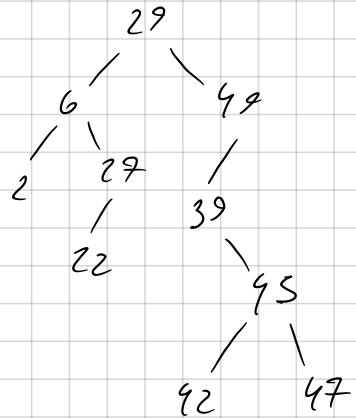
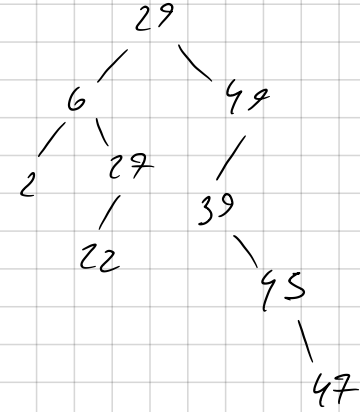
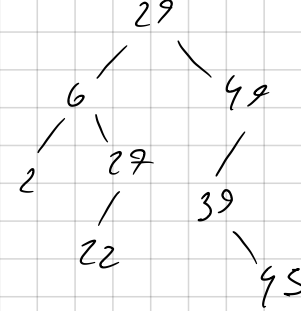
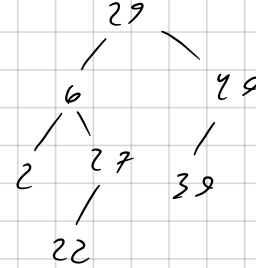
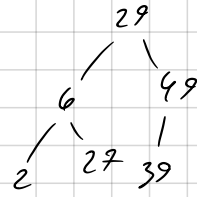
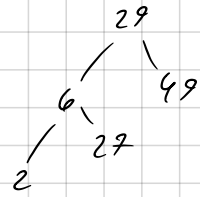
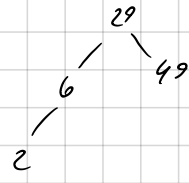
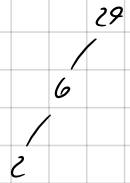
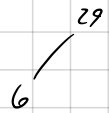
$T_{avg}(h) = \Theta(h)$

$T_{min}(h) = \Theta(1)$

4 Creazione albero

29 6 2 49 27 39 22 45 47 42

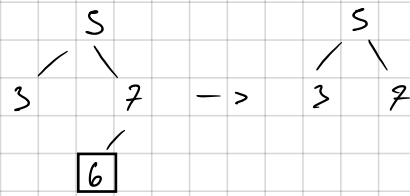
29



Concettazione

Strategie: Caso 1) **Nodo foglia (nessun genitore)**

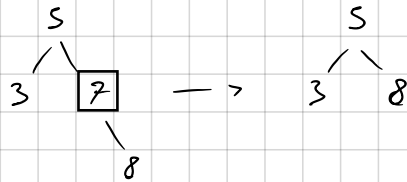
Il nodo viene semplicemente rimosso dal genitore senza dover riorganizzare l'albero



Eliminiamo il 6 e basta perché non ha figli

Caso 2) **Nodo con un solo figlio**

Il genitore del nodo da cancellare viene collegato direttamente all'unico figlio, si "salta" il nodo



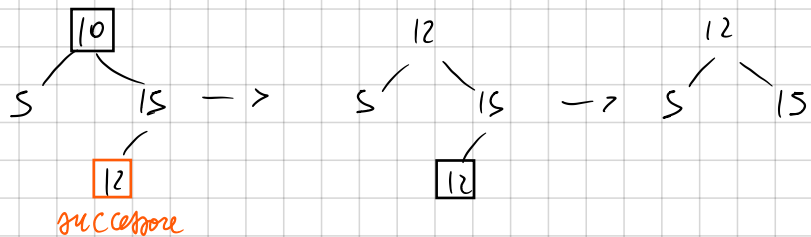
Cancelliamo il 7 e lo sostituiamo con l'8

Caso 3) **Nodo con 2 figli**

1) Si trova il successore in ordine (il più piccolo valore del sottoalbero destro) oppure il predecessore (il più grande nel sottoalbero sinistro)

2) Si sostituisce il valore del nodo da cancellare con quello del suo successore / predecessore

3) Poi si cancella il nodo duplicato (che sarà in uno dei primi due casi, quindi facile da gestire)



1) Trovo il successore

2) Sostituisco

3) Cancello il duplicato

procedura **BST_Delete** (T, Nodo z)

// Controllo se z è una foglia

if z.x == null and z.dx == null then

// Controllo se è la radice

if z.parent == null then

T.root = null

// Se z non è la radice allora è una foglia

else

if z.parent.x == z then

z.parent.x = null

else

z.parent.dx = null

// Se che z ha almeno un figlio

else

if z.x == null // ha figlio destra

Contrai_nodo (T, z, z.dx)

else // ha solo figlio sinistro o 2 figli

if z.dx == null then // solo sinistro

Contrai_nodo (T, z, z.x)

else // ha entrambi

succ = BST_succ (z)

z.Key = succ.Key

BST_Delete (succ)

procedura **contrai_nodo** (T, z, Child)

// Controllo se sta alla radice

if z.parent == null

T.root = Child

// Se non sono la radice allora ho figlio sinistro / destro

else

if z == z.parent.x

z.parent.x = Child

else

z.parent.dx = Child

if Child != null

Child.parent = z.parent

10

